Especificaciones — Ejercicio POO-08 aplicado a Counter-Strike 2

Asignatura	CYT646 — Lenguaje de Programación 3 (2026)
Alumno	Ariel Gaona
Ejercicio	POO-08 — CRUD REST, H2, reglas de juego y excepciones
Dominio	Counter-Strike 2 (simulación de **daño**)
Repositorio	https://github.com/ferue/agaona-taller-git-2026
Commit de la solución	https://github.com/ferue/agaona-taller-git-2026/commit/8b5e44e5a1dad762733e83b02bf0f025d1aa0e26

1. Objetivo

Publicar un servicio HTTP con Spring Boot sobre el modelado de Counter-Strike 2 que guarda su estado en una base **H2**. El servicio ofrece:

1. **Persistencia de tres clases del modelado:** la familia `Arma` (padre abstracto con las hijas `Rifle`, `Pistola`, `GranadaHumo` y `GranadaHE`), `Jugador` y `Posicion`.
2. **Un CRUD REST completo** sobre la familia `Arma` (`/api/armas`).
3. **Dos reglas de negocio del juego** que alimentan una **simulación persistida**: la explosión de una granada HE (`POST /api/explosiones`).
4. **Una jerarquía de excepciones** del dominio (una base y tres hijas) con **un handler HTTP por tipo**.
5. **Un protocolo de dominio** (`interface UICapable`, de POO-07) implementado por dos familias distintas y usado por `GET /api/hud`.
6. **Una colección de Insomnia** versionada en `insomnia/cs2-armas-crud.json`.

El servicio arranca con `./mvnw spring-boot:run` desde la carpeta `cs2/`.

2. Qué se persiste

Clase	Tabla(s)	Rol
`Arma` (abstracta) y sus hijas `ArmaDeFuego` → `Rifle`, `Pistola`; `Granada` → `GranadaHumo`, `GranadaHE`	`armas`, `armas_de_fuego`, `rifles`, `pistolas`, `granadas`, `granadas_humo`, `granadas_he`	Familia del CRUD. Herencia `JOINED`: una tabla por clase, unidas por `id`
`Jugador`	`jugadores`	Recibe el daño de la explosión
`Posicion` (`@Embeddable`)	columnas `pos_x`, `pos_y`, `pos_z` dentro de `armas` y `jugadores`	Mide distancias y calcula a dónde vuela un arma

Identidad: `id` numérico generado por H2 (`@GeneratedValue(strategy = IDENTITY)`). El JSON de creación **no** lo lleva. La respuesta del `POST` lo devuelve en el cuerpo y en la cabecera `Location`.

Subtipo en el JSON: al crear, el campo `"tipo"` elige la clase (`rifle`, `pistola`, `granada-humo`, `granada-he`). Al listar, `"tipo"` trae el nombre de la clase concreta (`"Rifle"`, `"GranadaHE"`...).

---

3. URLs

Base: `http://localhost:8080`

CRUD de la familia `Arma`

Método	URL	Cuerpo	Éxito	Errores
`GET`	`/`	—	200: servicio vivo y lista de endpoints	—
`POST`	`/api/armas`	`{"tipo", "nombre", "precio", "bando", ...}`	**201** + `id` + `Location`	400, 409
`GET`	`/api/armas` (opcional `?bando=CT`)	—	200: lista lo persistido	400 (bando inválido)
`GET`	`/api/armas/{id}`	—	200	**404**
`PUT`	`/api/armas/{id}`	igual que el `POST` (reemplaza el recurso entero)	200	400, 404, 409
`DELETE`	`/api/armas/{id}`	—	**204**	404

Campos opcionales del cuerpo:

- `skin` y `statTrak`;
- `cargadorCapacidad`, `danoBase` y `atributoEspecial` (mira o silenciador);
- `tiempoRetardo` y `duracionHumo`;
- `radioExplosion`, `danoMaximo` y `empujeMaximo`;
- `x`, `y`, `z`: el arma queda tirada en el mapa.

Jugadores, simulación y protocolo

Método	URL	Cuerpo	Éxito	Errores
`GET`	`/api/jugadores`	—	200	—
`GET`	`/api/jugadores/{id}`	—	200	404
`POST`	`/api/jugadores`	`{"nombre", "bando", "vida"?, "tieneChaleco"?, "x", "y", "z"}`	201	400, 409
`POST`	`/api/explosiones`	`{"granadaId", "x", "y", "z"}`	200: resultado de la simulación	400, 404, 409
`GET`	`/api/hud`	—	200: armas y jugadores como `UICapable`	—

Consola de la base

`http://localhost:8080/h2-console`, con JDBC URL `jdbc:h2:file:./data/cs2db`, usuario `sa` y contraseña vacía.

---

4. Invariantes

No hay setters públicos. El constructor vacío que exige JPA es `protected`. Todo cambio de estado pasa por constructores o mensajes que validan.

Clase	Invariante	Excepción	HTTP
`Arma`	`nombre`, `bando` y `skin` obligatorios	`CampoObligatorioException`	400
`Arma`	`precio ≥ 0`	`PrecioNegativoException`	400
`ArmaDeFuego`	`cargadorCapacidad > 0`	`CargadorInvalidoException`	400
`ArmaDeFuego`	`danoBase > 0`	`DanoInvalidoException`	400
`Granada`	`tiempoRetardo ≥ 0`	`TiempoRetardoInvalidoException`	400
`GranadaHumo`	`duracionHumo > 0`	`DuracionHumoInvalidaException`	400
`GranadaHE`	`radioExplosion > 0`, `danoMaximo > 0`, `empujeMaximo ≥ 0`	`RadioExplosionInvalidoException`, `DanoInvalidoException`, `EmpujeInvalidoException`	400
`GranadaHE`	explota una sola vez (se consume)	`GranadaYaDetonadaException`	409
`Jugador`	`0 ≤ vida ≤ 100`; `recibirDano` nunca la deja negativa	`VidaInvalidaException`	400
`Jugador`	siempre tiene posición	`CampoObligatorioException`	400
`Posicion`	coordenadas finitas	`CoordenadaInvalidaException`	400

Reglas del catálogo, en `ArmaService` y `JugadorService`:

Regla	Excepción	HTTP
Nombre de arma único (sin distinguir mayúsculas)	`ArmaDuplicadaException`	409
Precio ≤ $16.000 (dinero máximo de una partida competitiva)	`PrecioExcedidoException`	400
Bando del arma `T`, `CT` o `CT/T`; bando del jugador `T` o `CT`	`BandoInvalidoException`	400
Las granadas no admiten skins	`SkinNoPermitidaException`	400
Un `PUT` no cambia el tipo del arma	`CambioDeTipoException`	400
Nombre de jugador único	`JugadorDuplicadoException`	409

---

5. Las dos reglas y la simulación

Las dos reglas son **mensajes de `GranadaHE`**. El controller solo dispara la simulación, y en ella participan las tres clases: `GranadaHE`, `Jugador` y `Posicion`.

1. **Regla 1, daño (`danar(jugador, centro)`):** una granada HE daña a todos los jugadores vivos dentro de su radio, aliados incluidos. El daño baja con la distancia y el chaleco lo reduce a la mitad. Fórmula: `daño = danoMaximo × (1 − distancia / radio)`. Con chaleco, la mitad. La vida nunca baja de 0.
2. **Regla 2, empuje (`empujar(arma, centro)`):** toda arma tirada en el mapa dentro del radio sale despedida en dirección contraria a la explosión, y vuela más lejos cuanto más cerca estaba. Fórmula: `metros = empujeMaximo × (1 − distancia / radio)`. Si el arma estaba justo en el centro, sale hacia arriba.

Escenario de ejemplo (se carga solo con la base vacía; HE con id 6, radio 8 m, daño máximo 98, empuje máximo 6 m). `POST /api/explosiones` con `{"granadaId": 6, "x": 0, "y": 0, "z": 0}`:

Objeto	Distancia	Resultado esperado
Fénix (CT, sin chaleco)	1 m	recibe 86 → **14 de vida**
Halcón (CT, con chaleco)	5 m	37 a la mitad → **81 de vida**
Lobo (T, tenía 20 de vida)	2 m	74 de daño → **0, ELIMINADO**
Cuervo (T)	12 m	fuera del radio, **sin cambios**
Galil AR (tirada)	2 m	vuela 4,5 m → **(6.5, 0, 0)**
P250 (tirada en el centro)	0 m	sale 6 m hacia arriba → **(0, 0, 6)**
Tec-9 (tirada)	20 m	**no se mueve**
La granada HE	—	queda **DETONADA**; otro intento da 409

Todo queda persistido: `GET /api/jugadores`, `GET /api/armas`, `GET /api/hud` y la consola de H2 muestran el estado nuevo.

---

6. Jerarquía de excepciones

Todas son no comprobadas (`RuntimeException`). Así no hay que declararlas en cada capa, y `@Transactional` deshace los cambios si una regla falla.

```
Cs2Exception (abstracta)
├── DatoInvalidoException (abstracta) ........ 400  → 13 hijas (CampoObligatorio,
PrecioNegativo, VidaInvalida…)
├── RecursoNoEncontradoException (abstracta) . 404  → ArmaNoEncontrada, JugadorNoEncontrado,
TiendaNoEncontrada, ElementoUINoEncontrado
└── ReglaNegocioException (abstracta) ........ 400/409 → ArmaDuplicada, GranadaYaDetonada,
PrecioExcedido, BandoInvalido…
```

`ManejadorErrores` (`@RestControllerAdvice`) tiene **un `@ExceptionHandler` por cada excepción concreta**. Cada error responde:

```
{ "status": 409, "error": "Conflict", "tipo": "GranadaYaDetonadaException", "mensaje": "La granada 'HE Grenade' ya fue detonada y no se puede volver a usar" }
```

El campo `tipo` distingue a las hermanas aunque compartan el código HTTP.

---

7. Protocolo del dominio: `UICapable`

`UICapable` declara `obtenerIconoHud()`, `etiquetaHud()` y el método `default emitirSonidoInspeccion()`. Lo implementan dos familias distintas:

- **`Arma`**, con sus hijas: icono del arma, nombre con skin y sonido de inspección.
- **`Jugador`**: icono según bando y estado, nombre con vida o `ELIMINADO`, y sonido de radio.

`GET /api/hud` lee armas y jugadores de H2 y les habla solo como `UICapable`.

---

8. Cómo probarlo

Arranque

```
cd cs2
rm -rf data                 # base limpia (en Windows: borrar la carpeta cs2\data)
./mvnw spring-boot:run      # en Windows: .\mvnw.cmd spring-boot:run
```

Insomnia

1. **Importar:** en Insomnia, *Import* → *From File* → `insomnia/cs2-armas-crud.json`.
2. **Revisar el entorno:** `base_url` = `http://localhost:8080`, `id` = 1, `jugadorId` = 1 y `granadaId` = 6.
3. **Recorrer las carpetas en orden:**

Paso del enunciado	Carpeta y pedido	Qué se ve
1. `GET /` vivo	1 · *Servicio vivo*	200, `"estado": "OK"`
2. `POST` válido	1 · *Crear rifle (AWP)*	201 con `id`
3. `GET` colección	1 · *Listar todas*	el arma aparece
4. `GET` por `id`	1 · *Obtener por id*	coincide
5. Actualización	1 · *Actualizar (PUT)* y otra vez *Obtener por id*	precio 2500 y skin `StatTrak™ Vulcan` persistidos
6. `POST` ilegal	2 · *Precio negativo*, *Cargador 0*… y al final *Contar armas*	400 y la colección sigue con 14 armas
7. `DELETE`	9 · *Eliminar* y *Obtener el arma borrada*	204 y después 404
8. Endpoint del protocolo	7 · *HUD (GET /api/hud)*	armas y jugadores con icono, etiqueta y sonido
9. Simulación	5 · *Explotar la HE en (0, 0, 0)* y *Ver jugadores / armas después*	vidas y posiciones de la tabla de la sección 5

Paso del enunciado	Carpeta y pedido	Qué se ve
10. Cada excepción	carpetas 2, 4, 6 y 8	un pedido por excepción; el nombre indica el código y el `tipo` esperado

Consola H2

Abrir `http://localhost:8080/h2-console` y consultar:

```
SELECT * FROM ARMAS;
SELECT * FROM JUGADORES;
SELECT a.nombre, a.pos_x, a.pos_y, a.pos_z FROM ARMAS a WHERE a.pos_x IS NOT NULL;
SELECT a.nombre, h.detonada FROM ARMAS a JOIN GRANADAS_HE h ON h.id = a.id;
```

Después de la simulación, `JUGADORES.VIDA` y las posiciones de las armas muestran los valores nuevos.

Tests automáticos

```
./mvnw test     # 55 tests: dominio, services con H2 en memoria, controllers y handlers
```

`ManejadorErroresTest` comprueba que cada excepción concreta de la jerarquía tenga su propio handler.

---

9. Dónde está cada consigna

Consigna	Dónde
JPA + H2 + consola	`cs2/pom.xml`, `application.properties` (cada propiedad explicada en el README)
Tres clases persistidas; dos hijas «es un»; método abstracto sobrescrito	`domain/`: `Arma` → `Rifle`, `Pistola`, `GranadaHumo`, `GranadaHE`; `Jugador`; `Posicion`; `ejecutarAccionPrincipal()` y `obtenerDetalles()`
Constructor vacío de JPA y ocultamiento	constructores `protected` sin argumentos y sin setters públicos
Interfaz de dominio usada por un endpoint	`UICapable` ← `Arma`, `Jugador`; `GET /api/hud`
Repositorio como interfaz con consultas propias	`repository/ArmaRepository`, `repository/JugadorRepository`
CRUD REST	`rest/controller/ArmaCrudController`
Dos reglas en el dominio y simulación persistida	`GranadaHE.danar(...)`, `GranadaHE.empujar(...)`; `POST /api/explosiones`
Jerarquía de excepciones y handler por tipo	`exception/`; `rest/ManejadorErrores`
Colección de Insomnia	`insomnia/cs2-armas-crud.json` (58 pedidos, 9 carpetas)
Diagrama, reglas, «Qué hay que pensar»	`README.md`

Consigna	Dónde
Bitácora de IA	`BITACORA.md`